

TRISHIKA TRADING

Complete Code Backup

Date: 2026-04-11 04:56

PRODUCTION CONFIGS

ETH daily: ls_sma100, don=20, atr_len=14, atr_mult=2.0, sma=100, risk_pct=0.005

OOS: 33% profit rate, 0.35% mean DD

SOL daily: short_only, don=20, atr_len=14, atr_mult=2.0, risk_pct=0.005

OOS: 67% profit rate, ~2.3% mean DD

Regression: equity=979.06, trades=13

Daily command: bash healthcheck.sh

NEW PC SETUP:

1. pip install pandas numpy fpdf2
2. npm install
3. python test_regression.py (must PASS)
4. pm2 start ecosystem.config.cjs
5. bash healthcheck.sh

Re-create .env manually:

EXECUTION_ENABLED=true

EXECUTION_MODE=paper

ALERT_WEBHOOK_URL=<your webhook>

FILE INDEX:

- | | |
|---------------------------------------|------|
| 1. test_next_bar_strategy.py | [OK] |
| 2. test_regression.py | [OK] |
| 3. param_sweep.py | [OK] |
| 4. walk_forward.py | [OK] |
| 5. healthcheck.sh | [OK] |
| 6. run_backtest.py | [OK] |
| 7. src/strategy/donchian.js | [OK] |
| 8. src/strategy/live_signal_writer.js | [OK] |
| 9. src/strategy/monitor.js | [OK] |
| 10. server.js | [OK] |
| 11. ecosystem.config.cjs | [OK] |
| 12. start_trishika.sh | [OK] |
| 13. src/routes/crypto.js | [OK] |
| 14. src/candle/index.js | [OK] |

Page 2

TRISHIKA TRADING - CODE BACKUP

```

direction = 0          # +1 = long, -1 = short
entry_price = 0.0
stop_price = 0.0
highest_since_entry = 0.0 # used by chandelier long
lowest_since_entry = 0.0 # used by chandelier short
entry_i = entry_t = None
pending_exit_next_open = False
trades = []

for i in range(donchian_len + 1, len(df)):
    row = df.iloc[i]
    prev = df.iloc[i - 1]

    row_time = (
        row['time'] if 'time' in df.columns else
        row['timestamp'] if 'timestamp' in df.columns else i
    )

    # ?? pending next-open exit ?????????????????????????????????????????????????????????????
    if in_pos and pending_exit_next_open:
        exit_price = float(row['open'])

    if direction == 1:
        gross_pct = exit_price / entry_price - 1.0
        stop_dist = entry_price - stop_price if stop_price else None
        entry_px = entry_price
    else:
        gross_pct = entry_price / exit_price - 1.0
        stop_dist = stop_price - entry_price if stop_price else None
        entry_px = entry_price

    equity = _apply_pnl(equity, gross_pct, stop_dist, entry_px, None)

    trades.append({
        'entry_i': entry_i,
        'entry_t': entry_t,
        'entry_price': round(entry_price, 2),
        'exit_i': i,
        'exit_t': row_time,
        'exit_price': round(exit_price, 2),
        'direction': direction,
        'equity': round(equity, 2),
    })

    in_pos = False
    direction = entry_price = stop_price = 0
    highest_since_entry = lowest_since_entry = 0.0
    entry_i = entry_t = None
    pending_exit_next_open = False
    continue

# ?? no position: check entries ?????????????????????????????????????????????????????????????
if not in_pos:
    indicators_valid = (
        pd.notna(prev['upper_band']) and
        pd.notna(prev['lower_band']) and
        pd.notna(prev['atr'])
    )
    if not indicators_valid:
        continue

    sma_valid = pd.notna(prev['sma_filter']) if use_sma_filter else True
    above_sma = sma_valid and float(prev['close']) > float(prev['sma_filter'])
    below_sma = sma_valid and float(prev['close']) < float(prev['sma_filter'])
    sma_long_ok = (not use_sma_filter) or above_sma
    sma_short_ok = (not use_sma_filter) or below_sma

```

TRISHIKA TRADING - CODE BACKUP

```
long_signal = allow_long and sma_long_ok and prev['high'] > prev['upper_band']
short_signal = allow_short and sma_short_ok and prev['low'] < prev['lower_band']

# long takes priority if both fire simultaneously
if long_signal:
    in_pos = True
    direction = 1
    entry_price = float(row['open'])
    stop_price = entry_price - atr_mult * float(prev['atr'])
    highest_since_entry = float(prev['high'])
    lowest_since_entry = float(prev['low'])
    entry_i = i
    entry_t = row_time

elif short_signal:
    in_pos = True
    direction = -1
    entry_price = float(row['open'])
    stop_price = entry_price + atr_mult * float(prev['atr'])
    highest_since_entry = float(prev['high'])
    lowest_since_entry = float(prev['low'])
    entry_i = i
    entry_t = row_time

# ?? in position: manage stop ?????????????????????????????????????????????
else:
    if direction == 1:
        # ?? long stop management ?????????????????????????????????????????
        if chandelier:
            highest_since_entry = max(highest_since_entry, float(row['high']))

        if row['low'] <= stop_price:
            if next_open_exit:
                pending_exit_next_open = True
            else:
                exit_price = min(float(row['open']), float(stop_price))
                gross_pct = exit_price / entry_price - 1.0
                stop_dist = entry_price - stop_price
                equity = _apply_pnl(equity, gross_pct, stop_dist, entry_price, None)

                trades.append({
                    'entry_i': entry_i,
                    'entry_t': entry_t,
                    'entry_price': round(entry_price, 2),
                    'exit_i': i,
                    'exit_t': row_time,
                    'exit_price': round(exit_price, 2),
                    'direction': direction,
                    'equity': round(equity, 2),
                })

            in_pos = False
            direction = entry_price = stop_price = 0
            highest_since_entry = lowest_since_entry = 0.0
            entry_i = entry_t = None
        else:
            if ratchet:
                if chandelier:
                    new_stop = highest_since_entry - float(row['atr']) * atr_mult
                else:
                    new_stop = float(row['close']) - float(row['atr']) * atr_mult
                if new_stop > stop_price:
                    stop_price = new_stop

    else:
```

TRISHIKA TRADING - CODE BACKUP

```
# ?? short stop management ?????????????????????????????????????????
if chandelier:
    lowest_since_entry = min(lowest_since_entry, float(row['low']))

if row['high'] >= stop_price:
    if next_open_exit:
        pending_exit_next_open = True
    else:
        exit_price = max(float(row['open']), float(stop_price))
        gross_pct = entry_price / exit_price - 1.0
        stop_dist = stop_price - entry_price
        equity = _apply_pnl(equity, gross_pct, stop_dist, entry_price, None)

        trades.append({
            'entry_i': entry_i,
            'entry_t': entry_t,
            'entry_price': round(entry_price, 2),
            'exit_i': i,
            'exit_t': row_time,
            'exit_price': round(exit_price, 2),
            'direction': direction,
            'equity': round(equity, 2),
        })

    in_pos = False
    direction = entry_price = stop_price = 0
    highest_since_entry = lowest_since_entry = 0.0
    entry_i = entry_t = None
else:
    if ratchet:
        if chandelier:
            new_stop = lowest_since_entry + float(row['atr']) * atr_mult
        else:
            new_stop = float(row['close']) + float(row['atr']) * atr_mult
        if new_stop < stop_price:
            stop_price = new_stop

return equity, trades
```

FILE: test_regression.py

"""

Canonical regression test for test_next_bar_strategy.py (production engine).

ENGINE SPEC (do not mix with grid_search.py prototype):

- Execution : next-bar open entry and exit
- ATR : Wilder/EWM true-range, alpha=1/14, min_periods=14, adjust=False
- Ratchet : row-based close (current bar close - atr_mult * current bar ATR)
- Fees : 0.15% per side, additive, compounding
- Capital : 1000.0 initial
- Dataset : data/BTCUSD1h.csv MD5=3b5e774fedf92bec3b61da4f240f2ee8

PRODUCTION CONFIGS (validated Jul 2023?Apr 2026):

ETH daily ? ls_sma100 regime switch:

donchian_len=20, atr_len=14, atr_mult=2.0, sma_len=100
 allow_long=True, allow_short=True, risk_pct=0.005
 OOS: 33% profit_rate, 0.35% mean_dd, 3 folds

SOL daily ? short_only:

donchian_len=20, atr_len=14, atr_mult=2.0
 allow_long=False, allow_short=True, risk_pct=0.005
 OOS: 67% profit_rate, ~2.3% mean_dd (sized), 3 folds

BNB daily ? excluded (insufficient OOS trade frequency)

RETIRED TARGET: 807.05 (grid_search.py prototype ? 750 capital, no fees,
 same-bar close entry, SMA high-low ATR, 8 trades)

"""

```
import hashlib
import pandas as pd
from test_next_bar_strategy import run_next_bar_strategy
```

```
EXPECTED_EQUITY = 979.06
EXPECTED_TRADES = 13
CSV_MD5 = "3b5e774fedf92bec3b61da4f240f2ee8"
EQUITY_TOLERANCE = 0.01 # cents
```

```
PARAMS = dict(
    compounding = True,
    fee = 0.0015,
    ratchet = True,
    chandelier = False,
    donchian_len = 10,
    atr_len = 14,
    atr_mult = 2.0,
    initial_equity = 1000.0,
    fixed_notional = 1000.0,
    next_open_exit = False,
)
```

```
EXPECTED_TRADES_DETAIL = [
    # (entry_i, exit_i, entry_price, exit_price)
    ( 23, 26, 67539.34, 67196.56),
    ( 42, 76, 67205.06, 69711.04),
    ( 77, 80, 71467.81, 69916.80),
    (100, 112, 70186.08, 69486.82),
    (121, 149, 70412.10, 72200.81),
    (182, 214, 71211.95, 73029.77),
    (220, 233, 74109.56, 74311.26),
    (246, 265, 73914.40, 73527.56),
    (298, 315, 70272.00, 69971.19),
    (325, 349, 70767.85, 70123.06),
    (380, 381, 68899.92, 67874.20),
```

TRISHIKA TRADING - CODE BACKUP

```
(386, 411, 70122.15, 70179.04),
(423, 437, 70840.75, 70709.96),
]

def check_csv_integrity(path):
    with open(path, "rb") as f:
        md5 = hashlib.md5(f.read()).hexdigest()
    assert md5 == CSV_MD5, (
        f"CSV fingerprint mismatch.\n"
        f"  expected : {CSV_MD5}\n"
        f"    got      : {md5}\n"
        f"  The dataset has changed ? regression results are not comparable."
    )

def test_production_engine():
    csv_path = "./data/BTCUSD1h.csv"

    # 1. Dataset integrity
    check_csv_integrity(csv_path)

    # 2. Run engine
    df = pd.read_csv(csv_path)
    equity, trades = run_next_bar_strategy(df, **PARAMS)

    # 3. Trade count
    assert len(trades) == EXPECTED_TRADES, (
        f"Trade count mismatch: expected {EXPECTED_TRADES}, got {len(trades)}"
    )

    # 4. Final equity
    assert abs(equity - EXPECTED_EQUITY) <= EQUITY_TOLERANCE, (
        f"Equity mismatch: expected {EXPECTED_EQUITY}, got {equity:.4f}"
    )

    # 5. Per-trade path (entry_i, exit_i, entry_price, exit_price)
    for k, (t, exp) in enumerate(zip(trades, EXPECTED_TRADES_DETAIL), 1):
        ei, xi, ep, xp = exp
        assert t['entry_i'] == ei, \
            f"Trade {k}: entry_i expected {ei}, got {t['entry_i']}"
        assert t['exit_i'] == xi, \
            f"Trade {k}: exit_i expected {xi}, got {t['exit_i']}"
        assert abs(t['entry_price'] - ep) <= EQUITY_TOLERANCE, \
            f"Trade {k}: entry_price expected {ep}, got {t['entry_price']}"
        assert abs(t['exit_price'] - xp) <= EQUITY_TOLERANCE, \
            f"Trade {k}: exit_price expected {xp}, got {t['exit_price']}"

    print(f"PASS equity={equity:.2f} trades={len(trades)}")
    print("All 13 trade entry/exit bars and prices matched.")

if __name__ == "__main__":
    test_production_engine()
```

FILE: param_sweep.py

"""

Parameter sweep over the frozen production engine.

Outputs: sweep_results.csv (keyed by param tuple + dataset MD5)

Engine: test_next_bar_strategy.run_next_bar_strategy

Fixed: fee=0.0015, compounding=True, next_open_exit=False,
chandelier=False, initial_equity=1000.0

Grid: donchian_len x atr_len x atr_mult

"""

import hashlib

import itertools

import pandas as pd

from test_next_bar_strategy import run_next_bar_strategy

?? dataset ???

CSV_PATH = "./data/BTCUSDT_1h.csv"

with open(CSV_PATH, "rb") as f:

CSV_MD5 = hashlib.md5(f.read()).hexdigest()

df_base = pd.read_csv(CSV_PATH)

?? grid ???

DONCHIAN_LENS = [5, 8, 10, 12, 15, 20, 25, 30]

ATR_LENS = [7, 10, 14, 20, 25, 30]

ATR_MULTS = [1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0]

FIXED = dict(

fee = 0.0015,

compounding = True,

next_open_exit = False,

chandelier = False,

ratchet = True,

initial_equity = 1000.0,

fixed_notional = 1000.0,

)

?? metrics ???

def compute_metrics(equity_curve, trades, initial=1000.0):

if not trades:

 return dict(final_eq=initial, n_trades=0, win_rate=0.0,
 avg_ret=0.0, max_dd=0.0, sharpe_proxy=0.0)

rets = []

for t in trades:

gross = t['exit_price'] / t['entry_price'] - 1.0

rets.append(gross - 2 * FIXED['fee'])

wins = sum(1 for r in rets if r > 0)

win_rate = wins / len(rets) * 100.0

avg_ret = sum(rets) / len(rets) * 100.0

max drawdown over equity snapshots

eq_series = [initial] + [t['equity'] for t in trades]

peak = eq_series[0]

max_dd = 0.0

for eq in eq_series:

peak = max(peak, eq)

dd = (peak - eq) / peak * 100.0

max_dd = max(max_dd, dd)

Sharpe proxy: mean(rets) / std(rets) (no risk-free rate)

if len(rets) > 1:


```

import statistics
    sharpe = (sum(rets)/len(rets)) / (statistics.stdev(rets) + 1e-12)
else:
    sharpe = 0.0

return dict(
    final_eq      = round(eq_series[-1], 4),
    n_trades      = len(trades),
    win_rate      = round(win_rate, 2),
    avg_ret       = round(avg_ret, 4),
    max_dd        = round(max_dd, 4),
    sharpe_proxy  = round(sharpe, 4),
)

# sweep ?????????????????????????????????????????????????????????????????????????????????????
grid = list(itertools.product(DONCHIAN_LEN, ATR_LEN, ATR_MULTS))
total = len(grid)
results = []

print(f"Sweeping {total} combinations on {CSV_PATH} MD5={CSV_MD5}")
print(f"{'#':>5} {'don':>4} {'atr_l':>5} {'mult':>5} "
      f"{'eq':>9} {'trades':>7} {'wr%':>6} {'avgret%':>8} "
      f"{'maxdd%':>7} {'sharpe':>7}")

for k, (don, atr_l, mult) in enumerate(grid, 1):
    equity, trades = run_next_bar_strategy(
        df_base.copy(),
        donchian_len = don,
        atr_len       = atr_l,
        atr_mult       = mult,
        **FIXED,
    )
    m = compute_metrics(None, trades)

    row = dict(
        donchian_len = don,
        atr_len       = atr_l,
        atr_mult       = mult,
        csv_md5        = CSV_MD5,
        **m,
    )
    results.append(row)

if k % 20 == 0 or k == total:
    print(f"{k:>5} {don:>4} {atr_l:>5} {mult:>5.1f} "
          f"{m['final_eq']:>9.2f} {m['n_trades']:>7} "
          f"{m['win_rate']:>6.1f} {m['avg_ret']:>8.4f} "
          f"{m['max_dd']:>7.3f} {m['sharpe_proxy']:>7.4f}")

# save ?????????????????????????????????????????????????????????????????????????????????????
df = pd.DataFrame(results).sort_values("final_eq", ascending=False)
df.to_csv("sweep_results.csv", index=False)

print(f"\nDone. {total} combos. Results saved to sweep_results.csv")
print("\n?? Top 15 by final equity ??")
print(out.head(15).to_string(index=False))
print("\n?? Top 15 by Sharpe proxy ??")
print(out.sort_values("sharpe_proxy", ascending=False).head(15).to_string(index=False))

```

FILE: walk_forward.py

"""

Walk-forward validation over four candidate parameter sets.

Structure:

- Fixed window sizes: IS=300 bars, OOS=100 bars
- Step: 100 bars (anchored rolling ? IS grows, OOS stays fixed)
- Alternatively: sliding (IS fixed, rolls forward) ? set ANCHORED=False

Candidates:

1. (don=10, atr=14, mult=3.5) ? sweep leader
2. (don=12, atr=14, mult=3.5) ? sweep runner-up
3. (don=10, atr=14, mult=2.0) ? frozen production baseline (control)
4. (don=30, atr=14, mult=3.5) ? long-Donchian robustness probe

"""

import hashlib

import pandas as pd

from test_next_bar_strategy import run_next_bar_strategy

?? config ???

CSV_PATH = "./data/BTCUSD1h.csv"

IS_BARS = 300 # in-sample window size

OOS_BARS = 100 # out-of-sample window size

STEP = 100 # bars to advance each fold

ANCHORED = False # False = sliding IS window; True = expanding IS

CANDIDATES = [

```
dict(label="don10_atr14_m3.5", donchian_len=10, atr_len=14, atr_mult=3.5),
dict(label="don12_atr14_m3.5", donchian_len=12, atr_len=14, atr_mult=3.5),
dict(label="don10_atr14_m2.0", donchian_len=10, atr_len=14, atr_mult=2.0), # control
dict(label="don30_atr14_m3.5", donchian_len=30, atr_len=14, atr_mult=3.5),
```

]

FIXED = dict(

```
fee=0.0015, compounding=True, next_open_exit=False,
chandelier=False, ratchet=True,
initial_equity=1000.0, fixed_notional=1000.0,
```

)

?? helpers ???

def run(df_slice, cand):

params = {k: v for k, v in cand.items() if k != 'label'}

equity, trades = run_next_bar_strategy(df_slice.copy(), **params, **FIXED)

if not trades:

```
return dict(equity=1000.0, n_trades=0, win_rate=0.0,
            max_dd=0.0, avg_ret=0.0, profitable=False)
```

rets = [t['exit_price']/t['entry_price'] - 1.0 - 2*FIXED['fee'] for t in trades]

wins = sum(1 for r in rets if r > 0)

eq_series = [1000.0] + [t['equity'] for t in trades]

peak, max_dd = eq_series[0], 0.0

for eq in eq_series:

peak = max(peak, eq)

max_dd = max(max_dd, (peak - eq) / peak * 100.0)

return dict(

```
equity = round(equity, 4),
n_trades = len(trades),
win_rate = round(wins / len(trades) * 100, 1),
max_dd = round(max_dd, 3),
avg_ret = round(sum(rets)/len(rets)*100, 4),
profitable= equity > 1000.0,
```

)

?? dataset ???

with open(CSV_PATH, "rb") as f:

```
md5 = hashlib.md5(f.read()).hexdigest()
```

Page 11

TRISHIKA TRADING - CODE BACKUP

```
print("\n?? OOS summary per candidate ??")
print(f"{'Candidate':<22} {'folds':>5} {'profit_rate':>11} "
      f"{'mean_OOS_eq':>12} {'mean_OOS_tr':>11} {'mean_OOS_dd':>11}")

for label, g in out.groupby('label', sort=False):
    profit_rate = g['oos_profitable'].mean() * 100
    print(f"{label:<22} {len(g):>5} {profit_rate:>10.0f}% "
          f"{g['oos_equity'].mean():>12.2f} "
          f"{g['oos_trades'].mean():>11.1f} "
          f"{g['oos_max_dd'].mean():>11.3f}%")

print(f"\nSaved walk_forward_results.csv  ({len(rows)} rows)")
```

FILE: healthcheck.sh

```
#!/bin/bash
cd ~/projects/trishika-trading

echo "=== PM2 ==="
pm2 resurrect 2>/dev/null || true
sleep 3
pm2 list

echo ""
echo "=== Last 10 monitor events ==="
tail -10 logs/monitor.jsonl | python3 -c "
import sys, json
for line in sys.stdin:
    try:
        d = json.loads(line.strip())
        print(f\"{d['ts']}[:19]}  {d['level']}:<5}  {d['code']}:<25}  {d.get('message','')[:60]}\")
    except: pass
"

echo ""
echo "=== Signal file ages ==="
python3 -c "
import os, time
for f in ['signals_live/ETH_USD_1d.json', 'signals_live/SOL_USD_1d.json',
          'signals_live/ETH_USD.json', 'signals_live/SOL_USD.json']:
    if os.path.exists(f):
        age = (time.time() - os.path.getmtime(f)) / 60
        status = 'OK' if age < 150 else 'STALE'
        print(f'  {f:<35}  age={age:.0f}min  {status}')
    else:
        print(f'  {f:<35}  MISSING')
"

echo ""
echo "=== Alert summary ==="
python3 -c "
import json
from collections import Counter
counts = Counter()
try:
    for line in open('logs/monitor.jsonl'):
        d = json.loads(line.strip())
        counts[d['level']+'/' +d['code']] += 1
except: pass
alerts = {k:v for k,v in counts.items() if k.startswith('ALERT')}
if alerts:
    print('Alerts:')
    for k,v in sorted(alerts.items()): print(f'  {k:<40}  {v}')
else:
    print('No alerts ? clean')
print(f'Total MONITOR_CYCLE: {counts.get(\"INFO/MONITOR_CYCLE\",0)}')
print(f'Total SIGNAL_FRESH: {counts.get(\"INFO/SIGNAL_FRESH\",0)}')
"

echo ""
echo "=== Regression ==="
python test_regression.py
```

FILE: run_backtest.py

```
import pandas as pd
from test_next_bar_strategy import run_next_bar_strategy

def main():
    df = pd.read_csv('./data/BTCUSDT_1h.csv')

    eq, trades = run_next_bar_strategy(
        df,
        compounding=True,
        fee=0.0015,
        ratchet=True,
        chandelier=False,
        donchian_len=10,
        atr_len=14,
        atr_mult=2.0,
        initial_equity=1000.0,
        fixed_notional=1000.0,
        next_open_exit=False,
    )

    print(f"Final Equity: {eq:.2f}")
    print(f"Total Trades: {len(trades)}")
    for t in trades[:5]:
        print(t)

if __name__ == "__main__":
    main()
```

FILE: src/strategy/donchian.js

```

/**
 * donchian.js
 * Pure Donchian breakout + ATR trailing stop signal.
 *
 * Config (production-validated on BTCUSDT/ETHUSDT/SOLUSDT/BNBUSDT 1h):
 *   donchianLen = 30
 *   atrLen      = 14
 *   atrMult     = 2.0
 *   smaLen      = 200 (regime gate)
 *   allowLong   = true
 *   allowShort  = true
 *
 * Returns:
 *   { signal, direction, confidence, reason, donchianHigh, donchianLow,
 *     atr, sma, price, stopPrice }
 */

// ?? Wilder RMA (matches Python engine exactly) ?????????????????????????????????
function rma(values, period) {
  if (values.length < period) return null;
  let r = values.slice(0, period).reduce((a, b) => a + b, 0) / period;
  for (let i = period; i < values.length; i++) {
    r = (r * (period - 1) + values[i]) / period;
  }
  return r;
}

function calcATR(candles, period) {
  if (candles.length < period + 1) return null;
  const trs = [];
  for (let i = 1; i < candles.length; i++) {
    const h = candles[i].high, l = candles[i].low, pc = candles[i - 1].close;
    trs.push(Math.max(h - l, Math.abs(h - pc), Math.abs(l - pc)));
  }
  return rma(trs, period);
}

function calcSMA(values, period) {
  if (values.length < period) return null;
  const slice = values.slice(-period);
  return slice.reduce((a, b) => a + b, 0) / period;
}

export function donchianSignal(candles, opts = {}) {
  const {
    donchianLen = 30,
    atrLen      = 14,
    atrMult     = 2.0,
    smaLen      = 200,
    allowLong   = true,
    allowShort  = true,
  } = opts;

  const minBars = Math.max(donchianLen, atrLen, smaLen) + 2;
  if (!candles || candles.length < minBars) {
    return { signal: 'NEUTRAL', direction: 0, confidence: 0,
      reason: `Need ${minBars} candles, have ${candles?.length ?? 0}` };
  }

  // prev = second-to-last closed bar (last bar may be incomplete)
  const prev = candles[candles.length - 2];
  const curr = candles[candles.length - 1];

  // Donchian bands (shift(1).rolling ? exclude current bar)

```

TRISHIKA TRADING - CODE BACKUP

```
// shift(1).rolling(donchianLen): window of donchianLen bars ending at prev's predecessor
const lookback = candles.slice(-(donchianLen + 2), -2);
const donchianHigh = Math.max(...lookback.map(c => c.high));
const donchianLow = Math.min(...lookback.map(c => c.low));

// ATR (Wilder RMA on true range)
const atr = calcATR(candles.slice(0, -1), atrLen);

// SMA regime gate
const closes = candles.map(c => c.close);
const sma = calcSMA(closes.slice(0, -1), smaLen);

if (atr === null) {
  return { signal: 'NEUTRAL', direction: 0, confidence: 0,
    reason: 'ATR not ready' };
}

const price = curr.open; // next-bar open (current bar's open)
const aboveSMA = sma === null || prev.close > sma;
const belowSMA = sma === null || prev.close < sma;

const longBreak = prev.high > donchianHigh;
const shortBreak = prev.low < donchianLow;

const longOK = allowLong && longBreak && aboveSMA;
const shortOK = allowShort && shortBreak && belowSMA;

if (longOK) {
  const stopPrice = price - atrMult * atr;
  return {
    signal: 'LONG',
    direction: 1,
    confidence: 72,
    reason: `Donchian upper breakout above ${donchianHigh.toFixed(2)}, ATR stop ${stopPrice.toFixed(2)}`,
    donchianHigh, donchianLow,
    atr: +atr.toFixed(4),
    sma: sma ? +sma.toFixed(2) : null,
    price: +price.toFixed(2),
    stopPrice: +stopPrice.toFixed(2),
  };
}

if (shortOK) {
  const stopPrice = price + atrMult * atr;
  return {
    signal: 'SHORT',
    direction: -1,
    confidence: 72,
    reason: `Donchian lower breakdown below ${donchianLow.toFixed(2)}, ATR stop ${stopPrice.toFixed(2)}`,
    donchianHigh, donchianLow,
    atr: +atr.toFixed(4),
    sma: sma ? +sma.toFixed(2) : null,
    price: +price.toFixed(2),
    stopPrice: +stopPrice.toFixed(2),
  };
}

return {
  signal: 'NEUTRAL',
  direction: 0,
  confidence: 40,
  reason: `No breakout. Donchian range [${donchianLow.toFixed(2)}, ${donchianHigh.toFixed(2)}]`,
  donchianHigh, donchianLow,
  atr: +atr.toFixed(4),
  sma: sma ? +sma.toFixed(2) : null,
  price: +curr.close.toFixed(2),
}
```


TRISHIKA TRADING - CODE BACKUP

```
    stopPrice: null,  
  };  
}
```

FILE: src/strategy/live_signal_writer.js

```

/**
 * live_signal_writer.js
 * Runs Donchian strategy on closed candles and writes to signals_live/.
 *
 * Two pipelines:
 * 1. Hourly (called on every closed candle from CandleEngine)
 *    - BTC, ETH, SOL, BNB
 *    - smaLen: 100, donchianLen: 30, atrMult: 2.0
 *
 * 2. Daily ETH (called on schedule via refreshDailyETH)
 *    - fetches last 300 daily bars from Binance REST
 *    - smaLen: 100, donchianLen: 20, atrMult: 2.0
 *    - writes signals_live/ETH_USD_1d.json
 *
 * Validated config:
 *   hourly: short-only edge on ETH/BNB/SOL in bear regime (Nov 2025?Apr 2026)
 *   daily:  ls_sma100 profitable on 100% of combos on ETH full-cycle (Jul 2023?Apr 2026)
 */

import fs      from 'fs';
import path    from 'path';
import https   from 'https';
import { donchianSignal } from './donchian.js';

const SIGNALS_DIR = path.resolve('signals_live');

// ?? hourly symbol ? file mapping ?????????????????????????????????????????????
const SYMBOL_FILE = {
  'BTC': 'BTC_USD.json',
  'ETH': 'ETH_USD.json',
  'SOL': 'SOL_USD.json',
  'BNB': 'BNB_USD.json',
};

const SOL_DAILY_FILE = 'SOL_USD_1d.json';

// ?? validated configs ?????????????????????????????????????????????????????????
const HOURLY_OPTS = {
  donchianLen: 30,
  atrLen:     14,
  atrMult:    2.0,
  smaLen:     100, // updated: ls_sma100 dominates on full-cycle daily data
  allowLong:  true,
  allowShort: true,
};

const DAILY_ETH_OPTS = {
  donchianLen: 20,
  atrLen:     14,
  atrMult:    2.0,
  smaLen:     100, // ls_sma100 validated: ETH daily mean_eq=2549, 100% profitable combos
  allowLong:  true,
  allowShort: true,
};

// ?? shared write helper ???????????????????????????????????????????????????????
function writeSignalFile(filename, symbol, timeframe, sig, lastCandle) {
  const payload = {
    meta: {
      symbol,
      timeframe,
      strategy: 'Donchian-ATR',
      asOf:     new Date().toISOString(),
    },
  },

```

TRISHIKA TRADING - CODE BACKUP

```

price: {
  last: lastCandle.close,
  atr: sig.atr ?? null,
},
signal: {
  action: sig.signal,
  direction: sig.direction,
  confidence: sig.confidence,
  stopPrice: sig.stopPrice,
  reason: sig.reason,
},
indicators: {
  donchianHigh: sig.donchianHigh ?? null,
  donchianLow: sig.donchianLow ?? null,
  atr: sig.atr ?? null,
  sma: sig.sma ?? null,
  regime: sig.direction === 1 ? 'Bullish-Breakout'
    : sig.direction === -1 ? 'Bearish-Breakdown'
    : 'Ranging',
},
};

if (!fs.existsSync(SIGNALS_DIR)) fs.mkdirSync(SIGNALS_DIR, { recursive: true });
fs.writeFileSync(path.join(SIGNALS_DIR, filename), JSON.stringify(payload, null, 2), 'utf8');

if (sig.direction !== 0) {
  console.log(`[DONCHIAN] ${symbol}/${timeframe} ? ${sig.signal} `+
    `conf=${sig.confidence} stop=${sig.stopPrice} ${sig.reason}`);
}
}

// ?? hourly writer (called from CandleEngine on candle close) ??????????????????
export function writeDonchianSignal(symbol, candles) {
  try {
    const file = SYMBOL_FILE[symbol];
    if (!file) { console.warn(`[DONCHIAN] No file mapping for symbol: ${symbol}`); return; }
    const sig = donchianSignal(candles, HOURLY_OPTS);
    writeSignalFile(file, `${symbol}/USD`, '1h', sig, candles[candles.length - 1]);
  } catch (err) {
    console.error(`[DONCHIAN WRITER] ${symbol}:`, err.message);
  }
}

// ?? Binance REST fetch (daily bars) ?????????????????????????????????????????
function fetchBinanceKlines(symbol, interval, limit) {
  return new Promise((resolve, reject) => {
    const url = `https://api.binance.com/api/v3/klines?symbol=${symbol}&interval=${interval}&limit=${limit}`;
    https.get(url, { timeout: 15000 }, (res) => {
      let data = '';
      res.on('data', chunk => data += chunk);
      res.on('end', () => {
        try {
          const rows = JSON.parse(data).map(r => ({
            time: r[0],
            open: parseFloat(r[1]),
            high: parseFloat(r[2]),
            low: parseFloat(r[3]),
            close: parseFloat(r[4]),
            volume: parseFloat(r[5]),
          }));
          resolve(rows);
        } catch (e) { reject(e); }
      });
    }).on('error', reject).on('timeout', () => reject(new Error('Binance fetch timeout')));
  });
}

```

TRISHIKA TRADING - CODE BACKUP

```
// ?? daily SOL refresh (short_only, validated 67% OOS profit rate) ??????????????
export async function refreshDailySOL() {
  try {
    const candles = await fetchBinanceKlines('SOLUSDT', '1d', 300);
    if (candles.length < 130) {
      console.warn(`[DONCHIAN DAILY] SOL: Not enough bars: ${candles.length}`);
      return;
    }
    const sig = donchianSignal(candles, {
      donchianLen: 20, atrLen: 14, atrMult: 2.0,
      smaLen: 200,          // sma not used (allowShort only)
      allowLong: false, allowShort: true,
    });
    writeSignalFile(SOL_DAILY_FILE, 'SOL/USD', '1d', sig, candles[candles.length - 1]);
  } catch (err) {
    console.error('[DONCHIAN DAILY SOL]', err.message);
  }
}

// ?? daily ETH refresh (call on schedule, e.g. every hour) ??????????????????
export async function refreshDailyETH() {
  try {
    // fetch 300 daily bars ? enough for smaLen=100 + donchianLen=20 + warmup
    const candles = await fetchBinanceKlines('ETHUSDT', '1d', 300);
    if (candles.length < 130) {
      console.warn(`[DONCHIAN DAILY] Not enough bars: ${candles.length}`);
      return;
    }
    const sig = donchianSignal(candles, DAILY_ETH_OPTS);
    writeSignalFile('ETH_USD_1d.json', 'ETH/USD', '1d', sig, candles[candles.length - 1]);
  } catch (err) {
    console.error('[DONCHIAN DAILY ETH]', err.message);
  }
}
```

FILE: src/strategy/monitor.js

```

import fs      from 'fs';
import path    from 'path';
import https   from 'https';
import http    from 'http';
import { URL } from 'url';
import {
  portfolioGate,
  openPosition,
  closePosition,
  portfolioSnapshot,
} from './portfolio.js';
import { sendAlert } from './alert.js';
import { routeOrder, paperEquitySnapshot } from './execution.js';

const SIGNALS_DIR = path.resolve('signals_live');
const LOGS_DIR    = path.resolve('logs');
const MONITOR_LOG = path.join(LOGS_DIR, 'monitor.jsonl');
const TRADE_LOG   = path.join(LOGS_DIR, 'donchian_trades.jsonl');

const THRESHOLDS = {
  staleSignalMs: 2.5 * 60 * 60 * 1000,
  maxDdAlert:   0.007,
  minWinRateAlert: 0.20,
  winRateWindow: 10,
  maxFlipsPerDay: 6,
};

const WEBHOOK_URL = process.env.ALERT_WEBHOOK_URL || null;

const state = {
  lastSignal: null,
  lastRegime: null,
  equity:     1000.0,
  peakEquity: 1000.0,
  trades:     [],
  flipsToday: 0,
  flipsDate:  null,
};

function logEvent(level, code, message, data = {}) {
  const record = { ts: new Date().toISOString(), level, code, message, ...data };
  if (!fs.existsSync(LOGS_DIR)) fs.mkdirSync(LOGS_DIR, { recursive: true });
  fs.appendFileSync(MONITOR_LOG, JSON.stringify(record) + '\n', 'utf8');
  const prefix = level === 'ALERT' ? '?' : level === 'WARN' ? '??' : '?? ';
  console.log(`${prefix} [MONITOR/${code}] ${message}`);
  if (level === 'ALERT' && WEBHOOK_URL) {
    sendWebhook(`${prefix} *${code}*: ${message}`).catch(() => {});
  }
}

async function sendWebhook(text) {
  try {
    const u    = new URL(WEBHOOK_URL);
    const body = JSON.stringify({ text });
    const opts = {
      hostname: u.hostname,
      path:     u.pathname + u.search,
      method:   'POST',
      headers:  { 'Content-Type': 'application/json', 'Content-Length': Buffer.byteLength(body) },
    };
    const lib = u.protocol === 'https:' ? https : http;
    await new Promise((res, rej) => {
      const req = lib.request(opts, r => { r.resume(); r.on('end', res); });
      req.on('error', rej);
    });
  }
}

```

TRISHIKA TRADING - CODE BACKUP

```
    req.write(body);
    req.end();
  });
} catch (e) {
  console.error('[MONITOR] Webhook failed:', e.message);
}
}

function appendTradeLog(record) {
  if (!fs.existsSync(LOGS_DIR)) fs.mkdirSync(LOGS_DIR, { recursive: true });
  fs.appendFileSync(TRADE_LOG, JSON.stringify(record) + '\n', 'utf8');
}

function rollingWinRate(trades, window) {
  if (trades.length < window) return null;
  const wins = trades.slice(-window).filter(t => t.pnl_pct > 0).length;
  return wins / window;
}

function currentDrawdown(equity, peak) {
  return peak > 0 ? (peak - equity) / peak : 0;
}

function checkFreshness() {
  const file = path.join(SIGNALS_DIR, 'ETH_USD_1d.json');
  if (!fs.existsSync(file)) {
    logEvent('ALERT', 'SIGNAL_FILE_MISSING', 'ETH_USD_1d.json not found in signals_live/');
    sendAlert({ level: 'ERROR', type: 'SIGNAL_FILE_MISSING',
      message: 'ETH_USD_1d.json not found in signals_live/',
      payload: {} }).catch(() => {});
    return null;
  }
  const ageMs = Date.now() - fs.statSync(file).mtimeMs;
  const ageMin = Math.round(ageMs / 60000);
  if (ageMs > THRESHOLDS.staleSignalMs) {
    logEvent('ALERT', 'SIGNAL_STALE',
      `ETH daily signal is ${ageMin}min old (threshold: ${THRESHOLDS.staleSignalMs/60000}min)`,
      { age_min: ageMin });
    sendAlert({ level: 'WARN', type: 'SIGNAL_STALE',
      message: `ETH daily signal is ${ageMin}min old`,
      payload: { age_min: ageMin, threshold_min: THRESHOLDS.staleSignalMs/60000 } }).catch(() => {});
  } else {
    logEvent('INFO', 'SIGNAL_FRESH', `ETH daily signal age: ${ageMin}min`);
  }
  return JSON.parse(fs.readFileSync(file, 'utf8'));
}

function checkRegime(signal) {
  const regime = signal?.indicators?.regime ?? 'unknown';
  const action = signal?.signal?.action ?? 'NEUTRAL';
  if (state.lastRegime !== null && state.lastRegime !== regime) {
    logEvent('INFO', 'REGIME_CHANGE',
      `Regime changed: ${state.lastRegime} ? ${regime} signal=${action}`);
  }
  state.lastRegime = regime;
  return { regime, action };
}

function checkFlips(action) {
  const today = new Date().toISOString().slice(0, 10);
  if (state.flipsDate !== today) { state.flipsDate = today; state.flipsToday = 0; }
  if (state.lastSignal !== null && state.lastSignal !== action) {
    state.flipsToday++;
    logEvent('INFO', 'SIGNAL_FLIP',
      `Signal flipped: ${state.lastSignal} ? ${action} (flips today: ${state.flipsToday})`);
    if (state.flipsToday > THRESHOLDS.maxFlipsPerDay) {
```

TRISHIKA TRADING - CODE BACKUP

```

    logEvent('ALERT', 'TURNOVER_ANOMALY',
      `${state.flipsToday} signal flips today exceeds threshold ? possible data issue`,
      { flips: state.flipsToday });
  }
}
state.lastSignal = action;
}

function checkDrawdown() {
  const dd = currentDrawdown(state.equity, state.peakEquity);
  if (dd > THRESHOLDS.maxDdAlert) {
    logEvent('ALERT', 'DRAWDOWN_BREACH',
      `Equity drawdown ${((dd*100).toFixed(2))}% exceeds alert threshold ${THRESHOLDS.maxDdAlert*100}%`,
      { equity: state.equity, peak: state.peakEquity, dd_pct: +(dd*100).toFixed(3) });
    sendAlert({ level: 'ERROR', type: 'DRAWDOWN_BREACH',
      message: `Equity drawdown ${((dd*100).toFixed(2))}% exceeds threshold`,
      payload: { equity: state.equity, peak: state.peakEquity, dd_pct: +(dd*100).toFixed(3) } }).catch(() => {});
  }
}

function checkWinRate() {
  const wr = rollingWinRate(state.trades, THRESHOLDS.winRateWindow);
  if (wr === null) return;
  if (wr < THRESHOLDS.minWinRateAlert) {
    logEvent('ALERT', 'WIN_RATE_DRIFT',
      `Rolling ${THRESHOLDS.winRateWindow}-trade WR ${((wr*100).toFixed(1))}% below alert threshold`,
      { win_rate: +(wr*100).toFixed(1), trades: state.trades.length });
  } else {
    logEvent('INFO', 'WIN_RATE_OK',
      `Rolling ${THRESHOLDS.winRateWindow}-trade WR: ${((wr*100).toFixed(1))}%`);
  }
}

export function runMonitorCycle() {
  try {
    const signal = checkFreshness();
    if (!signal) return;
    const { regime, action } = checkRegime(signal);
    checkFlips(action);

    // Portfolio gate ? runs on every active (non-NEUTRAL) signal
    if (action !== 'NEUTRAL') {
      const gate = portfolioGate({
        symbol: signal.meta?.symbol ?? 'ETH',
        risk_pct: 0.005,
        equity: state.equity,
      });
      if (!gate.allowed) {
        logEvent('WARN', 'PORTFOLIO_LIMIT_BREACH',
          `Entry blocked for ${gate.payload.symbol}: ${gate.reason}`,
          gate.payload);
        sendAlert({ level: 'WARN', type: 'PORTFOLIO_LIMIT_BREACH',
          message: `Entry blocked for ${gate.payload.symbol}: ${gate.reason}`,
          payload: gate.payload }).catch(() => {});
      } else if (gate.reason !== 'SYMBOL_ALREADY_OPEN') {
        logEvent('INFO', 'PORTFOLIO_GATE_PASS',
          `Entry permitted for ${gate.payload.symbol}`,
          gate.payload);
      }
    }
  }
  const snap = portfolioSnapshot(state.equity);
  const execSnap = paperEquitySnapshot();
  logEvent('INFO', 'MONITOR_CYCLE', 'Cycle complete', {
    action,
    regime,
    confidence: signal.signal?.confidence ?? 0,
  });
}

```

TRISHIKA TRADING - CODE BACKUP

```
stop_price:    signal.signal?.stopPrice ?? null,
eth_price:     signal.price?.last      ?? null,
equity:        state.equity,
peak_equity:   state.peakEquity,
dd_pct:        +(currentDrawdown(state.equity, state.peakEquity)*100).toFixed(3),
trades_logged: state.trades.length,
...snap,
...execSnap,
});
} catch (err) {
  logEvent('ALERT', 'MONITOR_ERROR', `Monitor cycle failed: ${err.message}`);
  sendAlert({ level: 'ERROR', type: 'MONITOR_ERROR',
    message: `Monitor cycle failed: ${err.message}`,
    payload: { stack: err.stack?.slice(0, 300) } }).catch(() => {});
}
}

export function recordTrade({ entry_price, exit_price, direction, ts, symbol = 'ETH' }) {
  closePosition(symbol);
  const pnl_pct = direction === 1
    ? exit_price / entry_price - 1.0
    : entry_price / exit_price - 1.0;
  state.trades.push({ ts, pnl_pct, direction });
  state.equity    *= (1 + pnl_pct * 0.5);
  state.peakEquity = Math.max(state.peakEquity, state.equity);
  const record = { ts, entry_price, exit_price, direction, pnl_pct: +pnl_pct.toFixed(6) };
  appendTradeLog(record);
  checkDrawdown();
  checkWinRate();
  logEvent('INFO', 'TRADE_RECORDED',
    `Trade closed: ${direction===1?'LONG':'SHORT'} entry=${entry_price} exit=${exit_price} pnl=${(pnl_pct*100).toFixed(2)}%`);
}
```


FILE: server.js

```

import express from 'express';
import path from 'path';
import { fileURLToPath } from 'url';
import routes from './src/routes/index.js';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
const port = 3000;

app.use(express.json());
app.use('/api', routes);

// Serve static frontend from ./public
app.use(express.static(path.join(__dirname, 'public')));

import { refreshDailyETH, refreshDailySOL } from './src/strategy/live_signal_writer.js';
import { runMonitorCycle } from './src/strategy/monitor.js';

app.listen(port, () => {
  console.log(`? SERVER: http://localhost:${port}`);
  console.log(`? Signal: http://localhost:${port}/api/signal?symbol=BTC`);

  // Daily ETH signal refresh ? runs immediately then every hour
  // Uses nested setTimeout to avoid overlap if fetch takes longer than interval
  const DAILY_REFRESH_MS = 60 * 60 * 1000; // 1 hour
  async function scheduleDailyRefresh() {
    await refreshDailyETH();
    await refreshDailySOL();
    setTimeout(scheduleDailyRefresh, DAILY_REFRESH_MS);
  }
  scheduleDailyRefresh().catch(e => console.error('[SCHEDULER]', e.message));

  const MONITOR_MS = 5 * 60 * 1000;
  function scheduleMonitor() {
    runMonitorCycle();
    setTimeout(scheduleMonitor, MONITOR_MS);
  }
  scheduleMonitor();
});

```

FILE: ecosystem.config.cjs

```
module.exports = {
  apps: [{
    name:      'trishika',
    script:    'server.js',
    cwd:       '/home/himanshu/projects/trishika-trading',
    interpreter: 'node',
    node_args: '--experimental-vm-modules',
    instances:  1,
    autorestart: true,
    watch:      false,
    max_memory_restart: '400M',
    restart_delay: 5000,
    max_restarts: 10,
    env: {
      NODE_ENV:      'production',
      EXECUTION_ENABLED: 'false',
      EXECUTION_MODE: 'paper',
    },
    error_file: 'logs/pm2_error.log',
    out_file:   'logs/pm2_out.log',
    log_date_format: 'YYYY-MM-DD HH:mm:ss',
  }],
};
```

FILE: start_trishika.sh

```
#!/bin/bash
# start_trishika.sh
# Called by Windows Task Scheduler on login/boot
# Starts PM2 and resurrects the trishika process

export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"

cd ~/projects/trishika-trading

# Resurrect saved PM2 process list
pm2 resurrect 2>/dev/null || pm2 start ecosystem.config.cjs

# Save process list so next resurrect works
pm2 save

echo "[$(date)] trishika started" >> ~/projects/trishika-trading/logs/startup.log
```

FILE: src/routes/crypto.js

```

import express from "express";
import fs      from "fs";
import path    from "path";

const router = express.Router();

const SIGNALS_DIR = path.resolve("signals_live");

const SYMBOL_FILE = {
  BTC: "BTC_USD.json",
  ETH: "ETH_USD.json",
  SOL: "SOL_USD.json",
  BNB: "BNB_USD.json",
};

// GET /api/crypto/signal?symbol=BTC
router.get("/signal", (req, res) => {
  const symbol = (req.query.symbol || "BTC").toUpperCase();
  const file   = SYMBOL_FILE[symbol];

  if (!file) {
    return res.status(404).json({ error: `Unknown symbol: ${symbol}` });
  }

  const filePath = path.join(SIGNALS_DIR, file);

  if (!fs.existsSync(filePath)) {
    return res.json({
      signal:    "NEUTRAL",
      confidence: 0,
      reason:    "Signal not yet generated ? waiting for first closed candle",
      _source:   "Pending",
      symbol,
    });
  }

  try {
    const raw    = fs.readFileSync(filePath, "utf8");
    const data   = JSON.parse(raw);
    return res.json({
      signal:    data.signal?.action      ?? "NEUTRAL",
      confidence: data.signal?.confidence ?? 0,
      reason:    data.signal?.reason      ?? "",
      stopPrice: data.signal?.stopPrice   ?? null,
      direction: data.signal?.direction   ?? 0,
      _source:   "Donchian-ATR",
      asOf:      data.meta?.asOf          ?? null,
      indicators: data.indicators          ?? {},
      price:     data.price                ?? {},
    });
  } catch (err) {
    console.error("[CRYPTO ROUTE]", err.message);
    return res.status(500).json({ error: "Failed to read signal file" });
  }
});

export default router;

```

FILE: src/candle/index.js

```

import { WebSocket } from "ws";
import { CONFIG, WS_TYPES, BINANCE_TO_SYMBOL, TWELVE_TO_SYMBOL } from "../config/index.js";
import { computeIndicators } from "../indicators/index.js";
import { writeDonchianSignal } from "../strategy/live_signal_writer.js";

export class CandleEngine {
  constructor(symbol, candleMs = null) {
    this.symbol = symbol; this.candleMs = candleMs || CONFIG.candleMs;
    this.candles = []; this.currentCandle = null; this.lastAnalysis = null;
    this.analysisTime = 0; this.analyzing = false;
    this.subscribers = new Set(); this.source = "binance";
  }

  onTick(price, qty, time) {
    const bucketTime = Math.floor(time / this.candleMs) * this.candleMs;
    if (!this.currentCandle || this.currentCandle.time !== bucketTime) {
      if (this.currentCandle) {
        this.candles.push({ ...this.currentCandle });
        if (this.candles.length > CONFIG.maxCandles) this.candles.shift();
        this.broadcast({ type: WS_TYPES.CANDLE_CLOSED, candle: this.currentCandle, symbol: this.symbol });
        writeDonchianSignal(this.symbol, this.candles);
      }
      this.currentCandle = { time: bucketTime, open: price, high: price, low: price, close: price, volume: qty };
    } else {
      this.currentCandle.high = Math.max(this.currentCandle.high, price);
      this.currentCandle.low = Math.min(this.currentCandle.low, price);
      this.currentCandle.close = price;
      this.currentCandle.volume += qty;
    }
    this.broadcast({ type: WS_TYPES.TICK, candle: this.currentCandle, symbol: this.symbol });
  }

  getIndicators() { return computeIndicators(this.candles); }
  broadcast(msg) {
    const data = JSON.stringify(msg);
    for (const ws of this.subscribers) { if (ws.readyState === WebSocket.OPEN) ws.send(data); }
  }
  subscribe(ws) { this.subscribers.add(ws); }
  unsubscribe(ws) { this.subscribers.delete(ws); }
}

export function connectBinance(engines, log) {
  const streams = Object.values(CONFIG.symbols).map(s => `${s}@trade`).join("/");
  const ws = new WebSocket(`wss://stream.binance.com:9443/stream?streams=${streams}`);
  ws.on("open", () => log.info("Binance WS connected"));
  ws.on("close", () => { log.warn("Binance WS closed ? reconnecting in 3s..."); setTimeout(() => connectBinance(engines, log), 3000); });
  ws.on("error", (e) => log.error("Binance WS:", e.message));
  ws.on("message", (raw) => {
    try {
      const { data } = JSON.parse(raw);
      if (!data || data.e !== "trade") return;
      const sym = BINANCE_TO_SYMBOL[data.s.toLowerCase()];
      if (sym && engines[sym]) engines[sym].onTick(parseFloat(data.p), parseFloat(data.q), data.T);
    } catch (e) { log.error("Binance parse:", e.message); }
  });
}

export function connectTwelveData(engines, log) {
  if (!CONFIG.twelveKey) { log.warn("Twelve Data key missing ? Gold & EURUSD disabled"); return; }
  const ws = new WebSocket("wss://ws.twelvedata.com/v1/quotes/price?apikey=" + CONFIG.twelveKey);
  ws.on("open", () => {
    log.info("Twelve Data WS connected");
    const symbols = Object.values(CONFIG.forexSymbols);
    ws.send(JSON.stringify({ action: "subscribe", params: { symbols: symbols.join(",") } }));
    log.info(`Twelve Data ? subscribed: ${symbols.join(", ")}`);
  });
}

```

TRISHIKA TRADING - CODE BACKUP

```
ws.on("close", () => { log.warn("Twelve Data WS closed ? reconnecting in 5s..."); setTimeout(() => connectTwelveData(e
ws.on("error", (e) => log.error("Twelve Data WS:", e.message));
ws.on("message", (raw) => {
  try {
    const msg = JSON.parse(raw);
    if (msg.event === "heartbeat") return;
    if (msg.event === "subscribe-status") { log.info("Twelve Data subscribed:", JSON.stringify(msg)); return; }
    if (msg.event === "price" && msg.price) {
      const sym = TWELVE_TO_SYMBOL[msg.symbol];
      if (sym && engines[sym]) engines[sym].onTick(parseFloat(msg.price), 1, msg.timestamp ? msg.timestamp * 1000 : Da
    }
  } catch(e) { log.error("Twelve Data parse:", e.message); }
});
}
```